Cheap batches, weak gate: batching binary read-gate decisions 16–90-way at `effort=none` does not measurably degrade them, and a purpose-conditioned read gate on header and structure alone is not yet a classifier

Aswin Ram Kalugasala Moorthy Byline carried into this render from submission/main.tex.

Scope: prepared from a closed 2026-09-30 measurement run. Single model, single corpus, single task family. Short note / workshop scope.

Abstract

We measure a **binary file-read gate**: a small decision model that answers `{"relevant": true|false}` for a proposed read of file *f* for task *t* with stated purpose *p*, conditioned only on the question, the purpose, the file path, a short header and a structure outline — never on the file body. On 1,800 (task, file) pairs drawn from a 90-file corpus and 20 tasks, we test whether those per-read decisions can be batched. Batching 16 or 90 decisions into a single request at `reasoning_effort=none` reduces the number of decision calls from 1,800 to 120 and 20 (15× and 90×) and prompt tokens from 631 K to 270 K and 261 K (2.3× and 2.4×) while F1 moves from 0.218 to 0.227 and 0.211 — a spread of 0.016, within ±0.01 of the single-decision baseline. On this task, corpus and decision model, batching binary gate decisions at `effort=none` did not measurably degrade decision quality. This is a narrower claim than "batching is free": published batch-prompting losses on question answering and natural-language inference are large (−4 to −18 points; e.g. BoolQ 90.6 % → 72.8 %, −17.8, at batch size ≥ 64 without repair), and the present result is a measured exception on one task shape, not a refutation.

The same run yields a **negative result about the gate itself**. Across five configurations the best F1 is **0.320**, and the most precise configuration reaches precision **0.418** at recall **0.233** — the most precise gate tested silently drops three of every four gold files. Raising `reasoning_effort` from `none` to `minimal` doubles recall (0.175 → 0.367) for roughly 30 % more tokens, which is consistent with effort selecting a *different* classifier rather than a cheaper one. The gate is measurable and purpose-conditioned by design, but on header and structure alone it is **not yet an accurate classifier**, and we report it as a negative finding rather than hiding it behind the batching result. We state plainly what a binary gate decision is here, what the n = 1,800 figure counts, what was dropped from this write-up and why, and which numbers could not be established.

---

1 Introduction

An agent that reads repository files one at a time spends most of its context budget on files that do not matter. A natural defence is a **read gate**: before any file content moves into the reasoning model's context, a cheaper decision model is asked whether this particular read, for this particular task, is likely to be relevant. The gate is purpose-conditioned — it is given the task's question, the stated purpose of the read, the file path, a short header and a structure outline — and it never sees the file body, so the gate itself never pays read cost. If such a gate worked, it would be the cheapest possible place to cut context: one binary verdict in front of an expensive read.

Two questions follow immediately, and they are different questions:

1. **Cost.** Running a gate means one decision per candidate read. A 90-file corpus and 20 tasks is 1,800 decisions for a single pass. Can those decisions be **batched** — many verdicts per model call — without paying for it in decision quality?
2. **Quality.** Is the gate, run one-decision-per-call, actually an *accurate* classifier of read relevance from header and structure alone?

This paper reports both, from one closed measurement run on 2026-09-30. The first question has a positive answer, tightly scoped: at `reasoning_effort=none`, batching 16 or 90 binary gate decisions into one request did not measurably degrade decision quality, while cutting calls 15–90× and prompt tokens better than 2×. The second question has a **negative** answer, and it is the more consequential one for anyone considering the architecture: batching preserves the quality of a classifier that is not yet good enough to deploy as a sole gate. The best F1 observed is 0.320; the most precise configuration has precision 0.418 at recall 0.233.

Nothing here is a general claim about batching. The literature finds a real and reproducible cost/accuracy trade: batched prompting buys roughly an order of magnitude in calls and tokens at a −4 to −18 point accuracy cost under naive batching, recoverable to about ±1 point only with permutation and ensembling machinery (Cheng et al., 2023; Lin et al., 2023). Our finding is the opposite sign on one task shape — **binary JSON verdicts**, where the model's output per item is a single bit rather than a generated answer — and we treat it as a measured exception worth reporting, not as a law. Sections 3 and 4 are written so a reader can see exactly what was measured; Section 6 lists what the measurement cannot support.

Contributions.

- A **measured batching result** for binary gate decisions: 1,800 → 20 calls (90×) and 631 K → 261 K prompt tokens (2.4×) at a 0.007 F1 change relative to the single-decision baseline, at `effort=none` (Section 4.1).
- A **measured negative result** for the gate: best F1 0.320, best precision 0.418 at recall 0.233, on a 120-of-1,800 gold base rate (Section 4.3).
- An **honest scoping** of both: one decision model, one 90-file corpus, one task family, n = 20 tasks; the end-to-end staged-extraction arm of the same run is explicitly dropped from this paper and the drop is stated in Section 6, not hidden.

What is not claimed. No accuracy *gain* from gating. No generality beyond this corpus and

model. No claim that batching is free in general. No claim about downstream task success, context cost, or prompt-injection exposure — the arm of the run that measured those is out of scope here (Section 6).

---

2 Related work

2.1 Deciding whether to call a tool, or whether to retrieve

The idea that a model should decide *whether* to act before acting is well established. **Toolformer** (Schick et al., 2023) trains a model to decide which APIs to call and when, keeping a call only when it reduces next-token loss; its own stated limitation is that tool cost is not part of the decision. The antecedent it names, **TALM** (Parisi et al., 2022), bootstraps the same self-supervised worth-calling objective. **Self-RAG** (Asai et al., 2023) makes the decision explicit and binary: the model first determines whether retrieving passages would help, emitting a retrieval token when P(Retrieve = Yes) passes a threshold, and separately marks each retrieved passage Relevant/Irrelevant. **Mallen et al.** (2022) retrieve non-parametric memories "only when necessary", explicitly framing the gate as a cost-versus-accuracy choice. **FLARE** (Jiang et al., 2023) applies the same idea at token level, retrieving when a generation step is low-confidence.

These are the direct ancestors of a binary relevance verdict. The difference here is the **unit of decision**: a passage against a query, or a tool against an intent — not a *file proposed for reading*, conditioned on the *purpose* of that read, judged from header and structure without opening the file.

2.2 A cheap classifier in front of an expensive pipeline

Adaptive-RAG (Jeong et al., 2024) is the closest structural precedent for the present design: "a classifier, which is a smaller LM" is staged in front of a larger pipeline and picks between no retrieval, single-step retrieval and iterative retrieval. **RouteLLM** (Ong et al., 2024) routes queries between a stronger and a weaker model, reporting cost savings up to 3.66× with a threshold-swept accuracy/cost curve; **FrugalGPT** (Chen et al., 2023) established that a cheap front decision is worth measuring on both axes, reporting up to 98 % cost reduction. Guard classifiers such as **GuardAgent** (Xiang et al., 2024) report pre-action gates with high accuracy, but the criterion is safety policy, not task relevance.

The pattern — small model, binary-ish verdict, in front of a bigger one — is therefore shipped and measured. What is absent is the *file-read* unit and the *purpose-conditioned* criterion.

2.3 Batching: the cost case and the accuracy case

Batching many decisions into one request has a measured cost/accuracy trade in the literature, and it runs both ways.

- **For batching. Batch Prompting** (Cheng, Kasai & Yu, 2023, EMNLP 2023 Industry Track) shows inference costs decreasing "almost inverse linearly" with the number of

samples per batch, up to 5× with six samples while achieving better or comparable performance across ten datasets. The same abstract carries the caveat that batch size and task complexity affect performance.

- **Against naive batching. BatchPrompt** (Lin et al., 2023) finds that "prompting with batched data in longer contexts will inevitably lead to worse performance, compared to single-data prompting", and that performance "is significantly correlated with the positions and order of the batched data". Its measured deltas against one-question-per-call are substantial: BoolQ 90.6 % → 72.8 % at batch size ≥ 64 with one voting round (−17.8 points), recovering to 86.3 % (−4.3) with more rounds; GSM8K 94.9 % → 89.1 % at batch size 32 (−5.8). With the paper's permutation-plus-ensembling fixes at batch size 32, accuracy lands within roughly ±1.2 points of one-call-per-question while using 9–16 % of the calls. The paper also documents a failure mode unique to batching — silently missing answers when indices or delimiters are mishandled ("15 generated answers when input batch size is 16").
- **Coupling.** *Efficient but Vulnerable* (Yue & Yao, 2025) shows that a batch shares one context, so one item can perturb the others; the risk is cross-talk rather than batch size alone.

2.4 The gap this work addresses

A source-code taxonomy of 13 coding-agent scaffolds (Rombaut, 2026) finds file relevance decided by keyword/regex search, repository maps, AST search, knowledge graphs, embeddings or free LLM tool choice — retrieval and ranking machinery, and no per-read binary classifier. We read this as a **searched absence**, not a systematic review: the claim is that no published measurement of a purpose-conditioned per-file-read binary relevance gate, or of batching that gate's decisions, was found. On that basis, the batching delta for binary read-gate decisions is newly measured here, not replicated from prior work.

---

3 Method

3.1 The gate task

For a task t (a natural-language question about the corpus) and a candidate file f, the gate returns a single object:

```
{"relevant": true | false}
```

The decision is conditioned on the tuple **(question, purpose, file path, header, structure outline)**. The header is a 300-character preview; the structure outline is derived from markdown headings or code declarations. **The gate never receives the file body**, by construction, so gate cost does not scale with file size.

A **binary gate decision** in this paper means exactly one such verdict for exactly one (task, file) pair: the model asserts or denies that this file is worth reading for this task, given that stated

purpose, on header and structure evidence alone. It is not a ranking, a shortlist, or a score — it is a veto-or-admit bit. This matters for interpretation: a gate that is *precise* will block reads; a gate that is *recall-oriented* will admit too much context.

3.2 Fixture and gold labels

- **Corpus:** 90 files (documentation, source, tests, evals, examples, tools, scripts and four root files). The "1.83 MB in total" figure previously recorded here is **not reproducible** and is withdrawn: the fixture lists the 90 paths with **no size fields**, and the 90 files measure **2,426,190 bytes (2.426 MB) in the pinned snapshot tree** and **2,886,664 bytes (2.753 MB) in the live tree**. No byte record exists for the 2026-09-30 run date, so corpus size is stated as a two-snapshot measurement, not as a single figure. **[correction 2026-10-15, UFIXGR7, from the raw-record re-verification.]**
- **Tasks:** 20 questions with **mechanically verified** answers, produced by `grep` over the corpus at fixture-build time, with a zero-miss constraint.
- **Gold label:** `gold(t, f)` holds iff *f*'s bytes contain *t*'s answer. There is no human judgement anywhere in the label path, and the harness **recomputes gold from the bytes on every run** rather than trusting a stored label. **[correction 2026-10-15, UFIXGR7: re-deriving the gold set from corpus content under the rule the fixture states yields 127 pairs in the pinned snapshot tree / 130 in the live tree, against 120 stored; the stored labels are therefore not reproducible from the shipped corpus, and the 120-pair count, the 6.7 % base rate and every precision/recall/F1 figure below inherit them. This is a validity limitation, stated in full in this record's internal gap log (entry G-25; an internal working record not included in the released material).]**
- **Evaluation set:** every task is paired with every file, so one gate configuration is evaluated on **20 × 90 = 1,800 (task, file) pairs**. This is an exhaustive pairing, not a sample. Of the 1,800 pairs, **120 are gold** — a **6.7 % base rate**. Because answers are redundant across files, a single task's answer may live in more than one file.

The n = 1,800 figure therefore counts *pairs*, not files and not calls: 20 tasks × 90 files, evaluated once per configuration. The number of model **calls** made to produce those 1,800 verdicts depends on the batch size, and that is exactly what Section 4.1 varies.

3.3 Batching protocol

Three batch sizes are compared at `effort=none`:

- **bs1** — one decision per call: 1,800 calls for 1,800 verdicts.
- **bs16** — up to 16 decisions packed into one request: 120 calls.
- **bs90** — up to 90 decisions in one request, i.e. an entire task's candidate list in a single call: 20 calls.

Each decision in a batch is answered as its own binary JSON verdict; the harness maps each verdict back to its (task, file) pair. Two properties of the protocol are load-bearing for reading the result:

- **Unparsed decisions are counted, never coerced.** A verdict that fails to parse becomes `pred = null` and lands in an `unparsed` counter. The failure mode BatchPrompt warns about — a batched call silently returning fewer answers than items submitted — would surface here as unparsed rows. In this run it did not: every gate configuration reports **0 unparsed** and **0 call errors**.
- **Batching packs decisions, not answers.** What is batched is 16 or 90 independent one-bit verdicts over a shared instruction, not 16 or 90 generated answers.

3.4 Models and run control

- **Decision model:** `deepseek-v4-flash`, with `reasoning_effort` set as labelled (`none` or `minimal`). Temperature 0, JSON mode on decisions.
- **Rows are resumable**, de-duplicated by `(mode, task, tag)`, run at 8-way concurrency; retries are recorded rather than hidden. The run produced **9,260 rows with 0 call errors and 0 unparsed decisions**.
- Every mode's errors are counted in the summary; nothing is silently coerced or dropped.

3.5 Pre-registered arms

The run was designed before execution as a small arm table, so the batching comparison is pre-registered rather than post hoc:

arm	question
G1 — gate, bs1 × {`none`, `minimal`}	is a single binary read-decision accurate?
G2 — gate, bs ∈ {16, 90} × {`none`, `minimal`}	does batching 16 or 90 decisions preserve accuracy and cut cost?

Metrics are precision, recall, F1 and accuracy against gold, plus calls, prompt tokens, completion tokens and milliseconds per decision.

3.6 A note on accuracy as a metric

Accuracy is reported by the harness but is **misleading on this fixture** and we do not use it as a headline. At a 6.7 % gold base rate, a gate that always answers `false` scores **0.933**. Precision, recall and F1 against gold are the metrics that carry information. We report accuracy only alongside that caveat, because it appears in the source tables.

---

4 Results

All figures below come from the 2026-09-30 run. Every quantitative statement is traced to a source file and line (exact-line accuracy 71/73; two locators — C48 and C65's second — were repointed to the exact line in a correction pass); where a figure could only be re-derived from the raw rows rather than read from the summary, that is stated.

4.1 Batching binary gate decisions at `effort=none` (R2 — the positive result)

configuration	calls	prompt tokens	F1
bs1, `none`	1,800	631 K	0.218
bs16, `none`	120 (**15× fewer**)	270 K (**2.3× fewer**)	0.227
bs90, `none`	20 (**90× fewer**)	261 K (2.4× fewer)	0.211

At `reasoning_effort=none`, batching 16 or 90 binary JSON decisions into one request loses **nothing measurable** on this task: F1 is 0.218, 0.227 and 0.211, a total spread of 0.016 and within ±0.01 of the single-decision baseline. Meanwhile the call count falls 15× and then 90×, and prompt tokens fall 2.3× and 2.4×. Re-derived from the raw rows, the prompt-token totals are 631,210 / 269,614 / 261,090, matching the rounded 631 K / 270 K / 261 K; all five gate configurations report zero errors and zero unparsed decisions across 9,000 gate rows.

This is the cleanest positive result of the run, and it sits in the gap both provenance briefs identified: the published batch-prompting accuracy losses are measured on QA and NLI batches, and **binary gate decisions do not show that degradation here** (n = 1,800 per configuration, 0 errors).

What this does not say. It does not say batching is free in general, and it does not say the gate is good. It says that on this fixture, at this effort level, batching 16–90-way changed decision quality by less than the reported tolerance while cutting the decision layer's cost by an order of magnitude. The baseline F1 is 0.218: what batching preserved is a weak classifier's performance, not a useful one.

F1 equality hides a changed operating point, and for a gate that matters. Re-deriving the confusion counts makes a caveat visible that the F1 column conceals. Going from bs1 to bs16 at `effort=none`, true positives rise 21 → 44 and false positives rise 52 → 224; the number of pairs the gate *admits* rises from 73 to 268. At bs90 it is 146. The three configurations sit at different points on the same F1 contour: bs16 admits far more, and pays for its higher recall (0.175 → 0.367) with lower precision (0.288 → 0.164). For a metric like F1 the trade looks neutral; for a gate it is not neutral at all, because a gate exists to *block* reads, and a configuration that admits 268 of 1,800 pairs is a different component from one that admits 73. Batch size at `effort=none` therefore changes the gate's operating point at roughly constant F1, and anyone deploying a batched gate must pin the threshold behaviour, not just the F1.

4.2 The other side of the same table: batching multiplies thinking cost

The batching result is specific to `effort=none`, and the same table shows why that qualifier is not cosmetic. The `bs90 × minimal` configuration consumes **1.69 M completion tokens** — 8.9× the completion tokens of the `bs1 × minimal` configuration — because the model reasons over all 90 files in each call. Its total is **1.952 M tokens**, or **1,084 tokens per decision**, against **457 per decision** for `bs1 × minimal`: batching multiplies thinking cost, it does not amortise it, when extended reasoning is switched on. And it does so while scoring *lower*: F1 0.299 at bs90/minimal versus 0.320 at bs1/minimal.

So the batching result is not "batching is cheap"; it is "batching is cheap **at `effort=none`**, where the per-item output is a single bit", and it inverts to *more* expensive per decision at `effort=minimal`.

4.3 The gate is a weak classifier at every setting tested (R1 — the negative result)

configuration	precision	recall	F1	accuracy*	calls	total tokens	tok/decision
bs1, `none`	0.288	0.175	**0.218**	0.916	1,800	642 K	357
bs1, `minimal`	0.284	0.367	**0.320**	0.896	1,800	822 K	457
bs16, `none`	0.164	0.367	**0.227**	0.833	120	290 K	161
bs90, `none`	0.192	0.233	**0.211**	0.883	20	308 K	171
bs90, `minimal`	**0.418**	0.233	0.299	0.927	20	1,952 K	1,084

* Accuracy is misleading at a 6.7 % base rate — an always-`false` gate scores 0.933. F1 and recall are the headline numbers.

Two readings matter more than the table:

1. **No configuration is deployable as a sole gate.** The best F1 anywhere is **0.320**. The best precision anywhere is **0.418**, and it comes at recall **0.233** — the most precise gate tested silently drops **three of every four gold files**. A gate that admits too much is a cost problem; a gate that blocks three quarters of the relevant files is a correctness problem, and this design is in the second regime.
2. **Effort moves recall, not just cost.** Raising `reasoning_effort` from `none` to `minimal` at bs1 doubles recall, 0.175 → 0.367, for roughly 30 % more total tokens (642 K → 822 K). A smoke test in the source material illustrates the same axis directly: on an identical prompt, `effort=none` answered `{"relevant": false}` for a file that was gold for its task, while `minimal` reasoned its way to `true`. This is consistent with `reasoning_effort=none` selecting a *different* classifier rather than a cheaper one — which is precisely why the batching result in Section 4.1 must be pinned to `effort=none`.

The gate's design is purpose-conditioned and measurable; on header and structure alone, it is **not yet accurate**. We report the recall limitation as a finding. The source material's own diagnosis is that the gate's recall is bounded by what it can see: an early 500-character-header variant rejected gold files whose answer sat deep in the file, which is why the structure outline was added mid-design. Preview depth remains a live variable, not a settled one.

4.4 Summary of the two results

	question	answer	confidence
R2	does batching 16–90 binary gate decisions at	no measurable degradation on this task (F1 0.218 → 0.227 → 0.211;	single model, single calls corpus, 1,800

	question	answer	confidence
	`effort=none` degrade them?	1,800 → 120 → 20; prompt tokens 631 K → 270 K → 261 K)	pairs/configuration, 0 errors
R1	is the gate an accurate classifier on header + structure?	no — best F1 0.320; best precision 0.418 at recall 0.233	single model, 1,800 pairs/configuration, 5 configurations

---

5 Discussion

Why batching may be benign here. The published batching penalty is measured on batches of *generated answers*: BoolQ, RTE, QQP and GSM8K batches ask the model to produce one answer per item, and the failure modes BatchPrompt identifies — long-context degradation, position sensitivity, missing answers — are failure modes of generating and delimiting many answers in one context. The object batched here is different in kind: 16 or 90 independent **one-bit JSON verdicts over a shared instruction**. There is far less to interfere, the output per item is a single boolean, and the harness counts any verdict that fails to parse. That is a plausible mechanism for why the measured penalty is absent, but it is a hypothesis about *why*, not an additional measurement — we did not ablate output format against batch size, so the explanation is offered as a conjecture a reviewer should discount accordingly.

The practical reading for a builder. If you run a per-read gate at `effort=none`, you can batch 16–90 decisions per call and keep the decision layer's cost an order of magnitude down, with no measurable quality change on a task of this shape. That is genuinely useful: a 1,800-decision pass becomes 20 calls. But the binding constraint is not cost. It is that the gate blocks three of four relevant files at its most precise setting. **Batching makes a weak gate cheap; it does not make it good.**

The negative is the more important half. The gate was the cheapest place in the architecture to cut context, and the measurement says it is not yet accurate enough to be the only decision layer. That is a finding about the *design intent* — a purpose-conditioned veto on header and structure alone — and it is reported as such. The source run also measured components that do not depend on the veto's accuracy (a staged shortlist, and a router that collapsed to a single extraction method), but those arms are out of scope for this paper and are not used to rescue the gate's result here.

Effort is a design variable, not a knob. The recall doubling from `none` to `minimal` — and the 8.9× completion-token blow-up when those extra tokens are spent over 90 files per call — together say that `reasoning_effort` changes what classifier you have, not merely what it costs. Any comparison across configurations must pin effort, as this run does.

---

6 Limitations

Scope of the measurement. One decision model (`deepseek-v4-flash`), one corpus (90 files; 2.426 MB measured in the pinned snapshot tree / 2.753 MB live — the earlier "1.83 MB" is withdrawn as unsupported), one task family (locate a value stated in a file), 20 tasks, 1,800 pairs per configuration. Nothing here supports a claim about other corpora, other task shapes, other model families, or about *why* the batching penalty is absent. A reviewer should read R2 as a measured exception on one task shape.

The removed arm, stated rather than silently omitted. The same run included an end-to-end public arm (a staged shortlist feeding a routed extraction layer, six arms × 20 tasks) and an injection-exposure measurement. **We scope those out of this paper and do not report their numbers.** The reason is substantive, not editorial: with n = 20 per arm the accuracy differences among the top four arms are not statistically separable — the sign tests are $p \geq 0.219$, i.e. coin flips — and the staged-extraction harness that produced the exposure result is not part of the design published here. Consequently **this paper makes no claim about downstream task accuracy, context-cost reduction, or injection exposure.** The `stage` (R3) and router (R4) arms of the run are likewise out of scope. Any of them would need its own, properly powered write-up.

No significance test on the batching delta. "Within ±0.01" is a descriptive tolerance chosen before writing, not a pre-registered non-inferiority margin, and no null hypothesis is tested against the F1 differences. With 1,800 pairs the per-point sampling error is not negligible; the honest statement is that the observed F1 spread (0.016 across the three `effort=none` configurations) is small relative to anything the literature calls a batching loss, and that a formal non-inferiority test was not run.

F1 is the wrong single summary for a gate. As Section 4.1 shows, the three `effort=none` configurations land at materially different operating points — 73, 268 and 146 admitted pairs out of 1,800 — at nearly equal F1. A gate's cost and risk depend on *which* pairs it admits, not only on the harmonic mean of precision and recall. The present comparison therefore establishes F1-stability under batching and does **not** establish operating-point stability, and a deployment decision needs the latter.

Token accounting between two tables. The R1 table reports *total* tokens (642 K for bs1/`none`) while the R2 table reports *prompt* tokens (631 K for the same configuration); the difference is completion tokens (~11 K). The two are consistent once the units are aligned, but the source run report presents them in adjacent tables without labelling the change, which a reader can easily misread as a discrepancy.

A reproducibility wart in the shipped summary. The run's summary file records `prompt_tok_total` for the two bs1 configurations but leaves it `null` for `gate16_none` and `gate90_none`. The R2 prompt-token figures for bs16 and bs90 are therefore **not** corroborated by the summary file; we re-derived them from the raw per-row rows (269,614 and 261,090), which do match the report's rounded 270 K and 261 K. A reader reproducing from the summary alone would find two nulls where the paper quotes numbers.

Label provenance. Gold is defined mechanically as "the file's bytes contain the task's answer", recomputed each run. That removes human label noise but inherits the definition's limits: byte-containment is a *sufficient* but not *necessary* relevance criterion, so a file that is genuinely useful without containing the literal answer is scored as non-gold. Gate precision and recall are both measured against this definition.

What a second model would settle, and did not. The gate's weakness may be a property of this one decision model rather than of the task. That question is identified as a follows-up in the source material but not run, so it stays open.

No end-to-end accuracy ceiling is offered here. Because the end-to-end arm is out of scope, this paper cannot say what task accuracy a batched gate would deliver; it can only speak to the gate's own decision quality and cost.

---

7 Conclusion

Batching binary read-gate decisions 16- and 90-way at `reasoning_effort=none` did not measurably degrade them on this fixture: F1 moved from 0.218 to 0.227 and 0.211 while calls fell from 1,800 to 120 and 20 and prompt tokens fell from 631 K to 270 K and 261 K, with zero unparsed decisions and zero call errors. As a statement about batching, this is deliberately narrow — a measured exception to the −4 to −18 point losses reported for batched QA/NLI prompting, on one task shape, one corpus and one decision model; not a general licence to batch.

The same run shows why the efficiency result should not be oversold. The most precise gate configuration tested reaches precision 0.418 at recall 0.233, dropping three of four gold files, and the best F1 anywhere is 0.320. A purpose-conditioned read gate on header and structure alone is not yet a classifier, and no amount of batching fixes that. The next measurement that matters is not a cheaper batched gate; it is a gate that admits the files that matter, and a second decision model to establish whether the weakness is the task's or the model's.

References

Numbered references with identifiers, and the verification status of each, are maintained in this record's internal reference notes (an internal working record not included in the released material). **Author names below are exactly the set the source material carries** — surnames only where the source gives surnames only, and no given-name initials, because the source does not carry them and initials would be reconstructed rather than sourced. Titles and identifiers are as the source records them.

1. Asai, Wu, Wang, Sil & Hajishirzi (2023). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.* arXiv:2310.11511. (Venue not verified.)
2. Chen, Zaharia & Zou (2023). *FrugalGPT: How to Use Large Language Models While*

Reducing Cost and Improving Performance. arXiv:2305.05176.

3. Cheng, Kasai & Yu (2023). *Batch Prompting: Efficient Inference with Large Language Model APIs.* EMNLP 2023 Industry Track; arXiv:2301.08721.
4. Jeong, Baek, Cho, Hwang & Park (2024). *Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity.* NAACL 2024; arXiv:2403.14403.
5. Jiang et al. (2023). *Active Retrieval Augmented Generation (FLARE).* EMNLP 2023; arXiv:2305.06983.
6. Lin, Diesendruck, Du & Abraham (2023). *BatchPrompt: Accomplish More with Less.* arXiv:2309.00384.
7. Mallen et al. (2022). *When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories.* ACL 2023; arXiv:2212.10511.
8. Ong, Almahairi, Wu, Chiang, Wu, Gonzalez, Kadous & Stoica (2024). *RouteLLM: Learning to Route LLMs with Preference Data.* ICLR 2025; arXiv:2406.18665.
9. Parisi, Zhao & Fiedel (2022). *TALM: Tool Augmented Language Models.* arXiv:2205.12255.
10. Rombaut (2026). *Inside the Scaffold: A Source-Code Taxonomy of Coding Agent Architectures.* arXiv:2604.03515.
11. Schick, Dwivedi-Yu, Dessì, Raileanu, Lomeli, Hambro, Zettlemoyer, Cancedda & Scialom (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools.* NeurIPS 2023; arXiv:2302.04761.
12. Xiang et al. (2024). *GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning.* ICML 2025; arXiv:2406.09187.
13. Yue & Yao (2025). *Efficient but Vulnerable: Benchmarking and Defending LLM Batch Prompting Attacks.* Findings of ACL 2025; arXiv:2503.15551.